

Tahmin Etmek Yerine İz Sürmek: Yazılım Sistemlerinde Borç, İletişim ve Anlamanın Birleşik Bir Çerçevesi

Ömer Faruk Yavuz

Mayıs 2026

Özet

Bu çalışma, yazılım sistemlerinin sürdürülebilirliğini tek bir ilkeye indirgemektedir: kötü yapılandırılmış bir sistemde hem insan hem yapay zekâ ajanı tahmin eder; iyi yapılandırılmış bir sistemde ise ikisi de iz sürer. Bu ilke dört görünümde ele alınmaktadır — teknik ve örgütsel borcun taksonomisi, iletişimde netliğin örgütsel bir kaldıraç olarak işlevi, kültürün beyan edilen değerler değil tolere edilen davranışlarla kurulması ve kavramsal modelin (domain dili, basit tasarım) kodun anlamını taşıyan asıl katman olduğu. Çalışma bu dört görünümü ortak bir mekanizmaya — sistemin bıraktığı iz ile ihtiyaç duyulan bilgi arasındaki boşluk — bağlayarak, borç, iletişim ve tasarımın aslında aynı sorunun farklı yüzeyleri olduğunu savunmaktadır.

Giriş

Bir yazılım sisteminde bir değişiklik yapıldığında, üç şeyin bilinmesi gerekir: değişikliğin nereden geldiği, neyi etkilediği ve nasıl doğrulanacağı. Bu bilgi sistemde açıkça mevcutsa, değişikliği yapan kişi (insan ya da yapay zekâ ajanı) iz sürer. Bu bilgi mevcut değilse, aynı kişi tahmin eder. Bu çalışmanın merkezi tezi, yazılım mühendisliğinin birçok görünüşte farklı sorununun — teknik borç, örgütsel iletişim, kurumsal kültür, kavramsal model zayıflığı — aslında bu tek ayrımın farklı görünümleri olduğudur.

Tez dört bölümde geliştirilmektedir. İlk bölüm, borcu (teknik ve örgütsel) izin kaybolduğu noktaların bir kataloğu olarak ele alır. İkinci bölüm, iletişimin bu izi ya güçlendiren ya zayıflatan bir mekanizma olduğunu gösterir. Üçüncü bölüm, kültürün — beyan edilen değil fiilen tolere edilen davranışların toplamı olarak — izin kurumsallaşıp kurumsallaşmadığını belirlediğini savunur. Dördüncü bölüm, en temel izin kavramsal model olduğunu, kodun ancak doğru adlandırılmış bir domain dili üzerine kurulduğunda kendi izini taşıyabildiğini gösterir.

1 Borç: İzin Kaybolduğu Noktaların Kataloğu

1.1 Teknik Borç

Cunningham'ın orijinal metaforunda teknik borç, hızlı teslimat için alınan ve zamanla faiziyle geri ödenmesi gereken bir tasarım kısayoludur. Bu çalışma, borcu tek bir kod kalitesi sorunu olarak değil, sistemin kendi hakkında taşıdığı bilginin eksikliği olarak yeniden çerçeveler. Bu çerçevede borç kategorileri, hepsi aynı soruya farklı açılardan işaret eder — *sistemde neyin nerede, neden ve hangi garantiyle var olduğu bulunabiliyor mu*:

Borç Türü	İz Kaybının Belirtisi
Yapısal borç	Modül sınırları belirsiz; sorumluluk hangi dosyada yaşıyor bilinmiyor.
Davranış borcu	Sistemin hata, boş veri veya yetkisiz erişim durumunda ne yapacağı tanımsız.
Spesifikasyon borcu	Bir özelliğin "bitti" sayılması için hangi kriterin karşılanması gerektiği yazılı değil.
Test borcu	Kritik davranışı hangi testin koruduğu bilinmiyor; test var ama neyi doğruladığı belirsiz.
Dokümantasyon borcu	Bir kararın <i>neden</i> alındığı yalnızca o kararı alan kişinin hafızasında.
Ajan deneyimi borcu	Dosya ve kavram adları niyeti taşıyor; bir yapay zekâ ajanı güvenle değişiklik yapabileceği sınırları göremiyor.

Bu tablonun önemli bir sonucu, ajan deneyimi borcunun yeni bir kategori değil, dokümantasyon ve isimlendirme borcunun üretken yapay zekâ bağlamındaki yansıması olmasıdır: bir insanı yanıltan belirsizlik, bir ajanı da yanıltır — fark, ajanın bu belirsizlikte tahmin üretme hızının insana göre çok daha yüksek olması, dolayısıyla hatanın da o kadar hızlı yayılabilesidir.

1.2 Örgütsel Borç

Aynı mantık, kod dışına da taşınabilir. Bir organizasyonda karar sahipliğinin, sorumluluk sınırlarının veya önceliklerin nerede olduğu bulunamıyorsa, bu da bir borçtur — yalnızca kodda değil, insanların zihninde tahmin üretilmesine yol açar. Bu borcun akademik temeli, teknik borç kadar sağlam değildir; kavram büyük ölçüde pratisyen söyleminde gelişmiştir ve burada kesin bir ölçüm çerçevesi değil, bir teşhis metaforu olarak kullanılmaktadır. Buna karşın belirtileri tutarlıdır: kararların yavaşlaması, tekrarlayan açıklamalar, "bu kimin işi" sorusunun sıklığı.

2 İletişim: İzi Güçlendiren veya Silen Mekanizma

İletişim kalitesi, örgütsel hızın doğrudan bir çarpanıdır — ancak bu çarpan iki yönlü çarpan. Edmondson'ın psikolojik güvenlik kavramı ve yön birliği (*alignment*) araştırmaları, tutarlı ve net iletişimin belirsizliği azaltarak koordinasyon maliyetini düşürdüğünü gösterir; aynı mekanizma zayıf işlediğinde iletişim borcuna dönüşür — kararların gerekçesi kaybolur, aynı açıklama tekrar tekrar istenir.

Bu çerçevede belirli dil kalıplarının işlevi netleşir: bir düşünceyi netleştirme ("asıl meseleyi netleştirelim"), riski erken adlandırma ("gözden kaçırdığımız risk şu"), soyut tartışmayı somut bir adıma indirme ("bir sonraki en küçük adım ne") ve kendi varsayımını sorgulamaya açma ("burada yanılıyor olabilirim") — bunların ortak işlevi, düşünceyi başka birinin izleyebileceği bir ize dönüştürmektir. Bu ifadeler bir retorik beceri değil, iz bırakma disiplini: konuşmacı yalnızca sonucu değil, sonuca giden gerekçeyi de görünür kılar.

Aynı ilke, yazılım mühendisliğinde tekrar eden gözlemsel düzenliliklerle (bazen "yasa" olarak anılır) somutlaşır. Bunların bir kısmı gerçek ampirik veya kuramsal temele sahiptir: Hofstadter'in "iş sandığından uzun sürer" gözlemi ve Brooks'un geç kalan projeye eleman eklemenin projeyi daha da geciktirebileceği önermesi (*The Mythical Man-Month*), yönetim bilimi literatüründe yerleşiktir; Conway'in organizasyon yapısının sistem mimarisine yansıdığı önermesi de ampirik olarak tekrar sınanmıştır. Bunların ortak paydası, planlama ve koordinasyonun kendi izini bırakmadığında sistemin (proje takvimi, ekip yapısı, mimari) bunu telafi edecek biçimde bozulmasıdır. Bazı diğer popüler "yasalar" ise (yazılımın zamanla şişmesi, çevrimiçi ortamlarda yanlış cevabın doğru cevaptan daha hızlı düzeltme çektiği gözlemi gibi) daha çok anekdotik pratisyen bilgeliğidir ve bu ayrımın gözetilmesi, çerçevenin bilimsel iddiasını gerçekçi tutar.

3 Kültür: Beyan Edilen Değil Tolere Edilen

Kültürün izi, yazılı değerlerde değil, tekrar eden ve müdahale görmeyen davranışlarda bulunur. Bir organizasyon saygılı iletişimi beyan edip buna aykırı davranışı görmezden geliyorsa, çalışanların öğrendiği gerçek norm beyan değil toleranstır. Bu gözlem, örgütsel davranış literatüründeki "gözlenen norm, beyan edilen normdan güçlüdür" bulgusuyla örtüşür ve doğrudan izin kaybolması sorununa bağlanır: bir davranış sürekli tekrar edip sonuç doğurmuyorsa, o davranışın "neden kabul edilebilir" olduğuna dair iz silinmiş demektir — yeni katılan biri bunu ancak tahmin ederek öğrenir.

Bu ilke teknik pratiklere doğrudan uzanır: birleştirilmeden önce test edilmeyen kod, dokümansız özellik geliştirme veya sürekli ertelenen teknik borç, hepsi aynı mekanizmayla — tekrarlanan ve müdahale görmeyen davranış — kültüre dönüşür. Kültür değişimi bu nedenle yeni değerlerin ilan edilmesiyle değil, iş akışlarının, performans kriterlerinin ve karar mekanizmalarının yeniden tasarlanmasıyla mümkündür; çünkü kültürün taşıyıcısı söylem değil, sistemin ödüllendirdiği ve cezalandırdığı davranış örüntüsüdür.

4 Kavramsal Model: İzin Kökeni

Önceki üç bölümün ortak ön koşulu, sistemin doğru kavramlarla düşünülmüş olmasıdır. Evans'ın alan güdümlü tasarım (*domain-driven design*) yaklaşımı, kodun teknik veri yapılarından önce iş alanının gerçek kavramlarıyla modellenmesi gerektiğini savunur; bu yaklaşımın merkezi aracı, geliştirici, ürün sahibi ve kodun aynı terimleri aynı anlamda kullandığı ortak dildir (*ubiquitous language*). Bir sistemde `summary`, `manualSummary` ve `generatedSummary` gibi teknik olarak benzer ama anlamca farklı alanlar aynı isim altında toplandığında, iz kaybolur: kodu okuyan biri (insan ya da ajan) hangi alanın hangi iş kuralına bağlı olduğunu tahmin etmek zorunda kalır.

Bu gözlem, kritik davranışların açık bir sözleşme (*behavior contract*) olarak ifade edilmesi gerektiği sonucuna varır — "el ile düzenlenmiş özet, yapay zekâ tarafından üretilen özeti üzerine yazılmamalıdır" gibi bir cümle, yalnızca bir test senaryosu değil, sistemin bozulmaması gereken bir izidir. Meyer'in sözleşmeye dayalı tasarım (*design by contract*) kavramındaki ön koşul, son koşul ve değişmez (*invariant*) ayrımı, bu izin biçimsel karşılığıdır.

Kavramsal netliğin ikinci bileşeni basitliktir. Beck'in basit tasarım için önerdiği dört kural — testlerin geçmesi, kodun niyeti açıkça göstermesi, tekrarın olmaması ve gereksiz yapının bulunmaması — sıralı bir öncelik taşır: anlaşılabilirlik, tekrarsızlıktan; tekrarsızlık, asgari yapıdan önce gelir. Bu sıralamanın işlevi, gelecekteki her ihtimali bugünden karşılayan karmaşık yapılar kurmak yerine, yalnızca şu an gerekli olanı inşa etmektir — çünkü her fazladan yapı, gelecekte izlenmesi gereken bir yüzey daha demektir. Bu ilke, Unix felsefesindeki "küçük olan iyidir, her program tek işi iyi yapmalıdır" önermesiyle (Raymond) aynı kökten beslenir: karmaşıklık, büyük yekpare sistemlerle değil, iyi tanımlanmış küçük parçaların hiyerarşik biçimde birleştirilmesiyle yönetilir.

Son olarak, doğru kavramların ortaya çıkması genellikle koddan önce, sistemin problem alanındaki olayları düz insan diliyle anlatabilmesiyle başlar. Bir sistemi "PaymentService yazacağız" diye değil, "bir öğrenci bir kursa erişim hakkı kazanır, bu erişim bir politikaya göre eğitime hak ediş üretir" diye anlatabilmek, kavramları (erişim hakkı, politika, hak ediş) koddan önce görünür kılar. Bu anlatı, teknik tasarımın üzerine kurulacağı ilk ve en kalıcı izdir.

5 Sonuç

Bu çalışma, yazılım mühendisliğinde görünüşte ayrı duran dört sorunu — teknik ve örgütsel borç, iletişim kalitesi, kurumsal kültür ve kavramsal model — tek bir mekanizmaya bağlamıştır: sistemin bıraktığı iz ile bu izi arayan tarafın (insan ya da yapay zekâ ajanı) ihtiyaç duyduğu bilgi arasındaki boşluk. Borç, bu boşluğun kod düzeyindeki birikimidir; zayıf iletişim, bu boşluğun insanlar arası düzeydeki üretimidir; tolere edilen davranış, bu boşluğun kurumsallaşmasıdır; kavramsal model zayıflığı ise boşluğun kökeninde yatan, en pahalı biçimdir — çünkü yanlış adlandırılmış bir kavram, üzerine kurulan her kod satırına, her teste ve her karara aynı belirsizliği miras bırakır.

Bu çerçevenin pratik sonucu tektir: bir sistemi iyileştirmenin en genel ölçütü, "bu sistemde bir sonraki değişikliği yapacak kişi tahmin mi eder, iz mi sürer?" sorusudur. Kod incelemesi, dokümantasyon, iletişim normu ve isimlendirme kararı — hepsi bu tek soruya verilen cevabı değiştirir.

Kaynaklar

- [1] W. Cunningham, "The WyCash Portfolio Management System," *OOPSLA Experience Report*, 1992.
- [2] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, Boston, 2003.
- [3] B. Meyer, *Object-Oriented Software Construction*, 2. baskı, Prentice Hall, Upper Saddle River, 1997.

- [4] A. C. Edmondson, “Psychological Safety and Learning Behavior in Work Teams,” *Administrative Science Quarterly*, vol. 44, no. 2, 1999.
- [5] F. P. Brooks Jr., *The Mythical Man-Month: Essays on Software Engineering*, Addison-Wesley, Reading, 1975.
- [6] M. E. Conway, “How Do Committees Invent?,” *Datamation*, vol. 14, no. 4, 1968.
- [7] K. Beck, *Extreme Programming Explained: Embrace Change*, 2. baskı, Addison-Wesley, Boston, 2004.
- [8] E. S. Raymond, *The Art of Unix Programming*, Addison-Wesley, Boston, 2003.